```
exports.fromDir = (root) ->

  return unless root

  compile = (path, callback) ->
    coffee = spawn 'coffee', ['-c', '-o', "#{root}compiled/#{path}", path]

    coffee.on 'exit', (code, s) ->
      if code is 0 then compileClient callback
      else console.log 'error compiling'

    coffee.on 'message', (data) ->
     console.log data

  createArtifact = (path, version, callback) ->
    execFile "tar", ["-cvf", "artifact.#{version}.tar", path], (e, d) ->
      callback?()

  runSpecs = (folder) ->
    forAllSpecsIn "#{root}#{folder}", (file) ->
      require "./#{file}"

  clean = (path, callback) ->
    exec "rm -r #{root}#{path}", (err) -> callback?()

  copy = (src, dst, callback) ->
    exec "cp -R #{root}#{src} #{root}#{dst}/.", ->
      callback?()

  runApp = (env) ->
    exec 'NODE_ENV=#{env} node compiled/app/server.js &', ->
      console.log "Running..."

  {clean, compile, copy, createArtifact, runSpecs, runApp}
```

Create the build artifact.

The Cakefile is complete, but you can see a callback waterfall in some of the tasks—where asynchronous operations that must happen in sequence lead you to deeply nested callbacks:

```
task 'build', 'run the build', ->
  clean 'compiled', ->
    compile 'app', ->
      copy 'content', 'compiled', ->
        createArtifact 'compiled', VERSION, ->
          console.log 'Build complete'
```

Compare this to a roughly equivalent section of a Makefile:

```
build: artifact

artifact: clean compile copy
    tar –cvf artifact.tar compiled
```

Cake is a simple tool, and in some instances you may be better off using a dedicated build tool. That said, CoffeeScript provides powerful syntactic